



2025-2026 Fall Semester

COMP413

Internet of Things

Final Project Report

Instructor

Dr. Abdulkadir KÖSE

Subject

Illegal Parking Detection

Submission Date

08.01.2026

Submitted by

Group 2

Syed Muhammad Mojiz Ali ZAIDI - 2311051006

Selin ERYAŞAR - 2111051051

Mohamed MOSTAFA - 2211011095

Güleser KABA-110510234

ILLEGAL PARKING DETECTION SYSTEM USING IOT AND TINYML

Abstract

Illegal parking is becoming a serious concern in urban areas, causing traffic congestion, safety hazards, and inefficient usage of infrastructure facilities. This work proposes an IoT-based illegal parking detection system that utilizes edge computing and TinyML for real-time vehicle presence detection. Each restricted parking location has an associated independent ESP32-based IoT node with a camera and a distance sensor. A lightweight machine learning model runs directly on the microcontroller for vehicle presence inference, while it uses MQTT for real-time communication with a backend server and monitoring dashboard. Overall, the proposed system provides low-latency detection, reduced network usage, and easy scalability for deployments in smart city applications.

1. INTRODUCTION

The rising number of people in urban areas and vehicles has exacerbated parking problems in current city infrastructure. Illicit parking practices contribute to route disruption of vehicles for emergency routes and pedestrian paths. Accordingly, this phenomenon inhibits traffic movement and causes inefficiency in the performance of urban transport infrastructure. It was shown in reference [1] that a lack of efficient parking practices results in deteriorated travel time utilization and infrastructure use in urban towns.

Conventional parking enforcement methods typically rely on either manual inspection or monitoring via cameras in a centralized mode. However, such methods are usually labor-intensive, expensive in terms of deployment, and hard to extend within large urban environments. Recent works underline that these kinds of centralized surveillance solutions are prone to show high latency, heavy network usage, and limited adaptability against dynamic city conditions [2]. These issues have currently shifted research toward distributed and automated monitoring solutions based on the Internet of Things (IoT) platform.

IoT-based smart parking systems utilize embedded sensors and microcontrollers with wireless communication protocols for the real-time monitoring of parking spaces. To date, vehicle detection has widely been conducted using ultrasonic and proximity sensors in combination. Light-weight communication protocols, such as MQTT, enable low-latency data transmission to a central server [3]. Such architectures allow continuous monitoring while minimizing infrastructure and operational costs.

For further improvement of detection accuracy, recent works have integrated the techniques of computer vision and embedded AI into parking monitoring. Edge-based machine learning provides vehicle classification on resource-constrained devices directly, reducing constant video streaming and cloud-side processing [4]. Among these, TinyML frameworks run neural network inference on microcontrollers by providing low latency, lower bandwidth usage, and better system scalability [5].

To address the mentioned issues and based on the developments that trigger the concept of this research, this study proposes an IoT-based illegal parking detection system incorporating ESP32-based edge nodes, the use of sensors for parking detection, vehicle verification using TinyML machine learning, and real-time interactions between the edge devices and the centralized server. The proposed system can efficiently make decisions through edge intelligent computation and send particular events to its server.

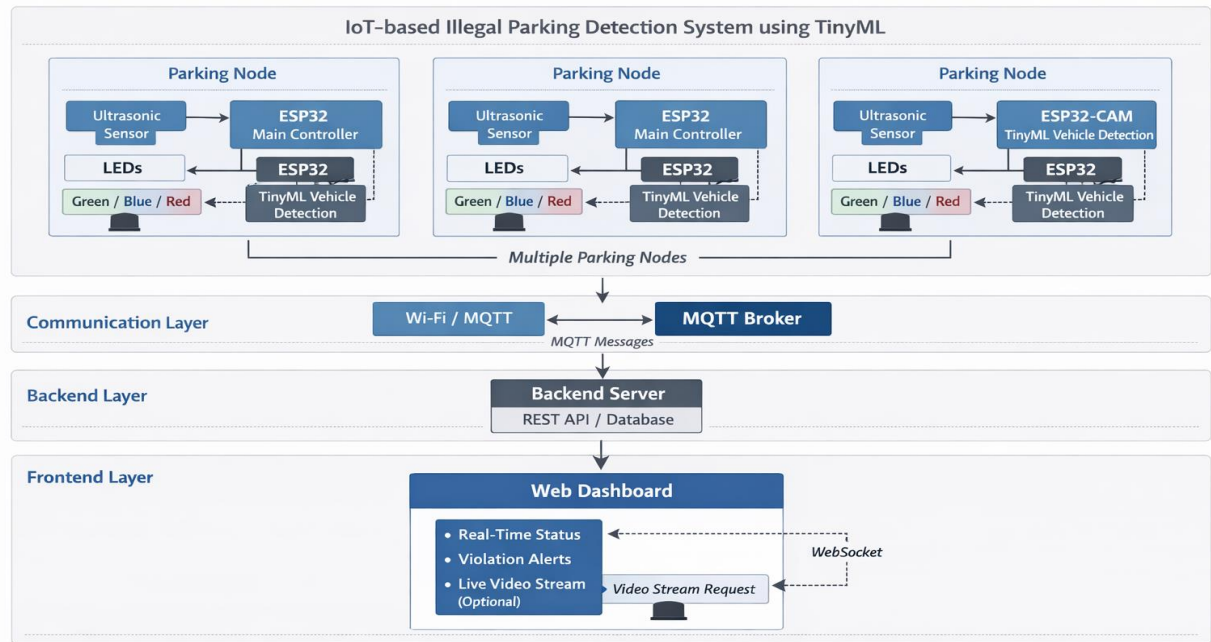


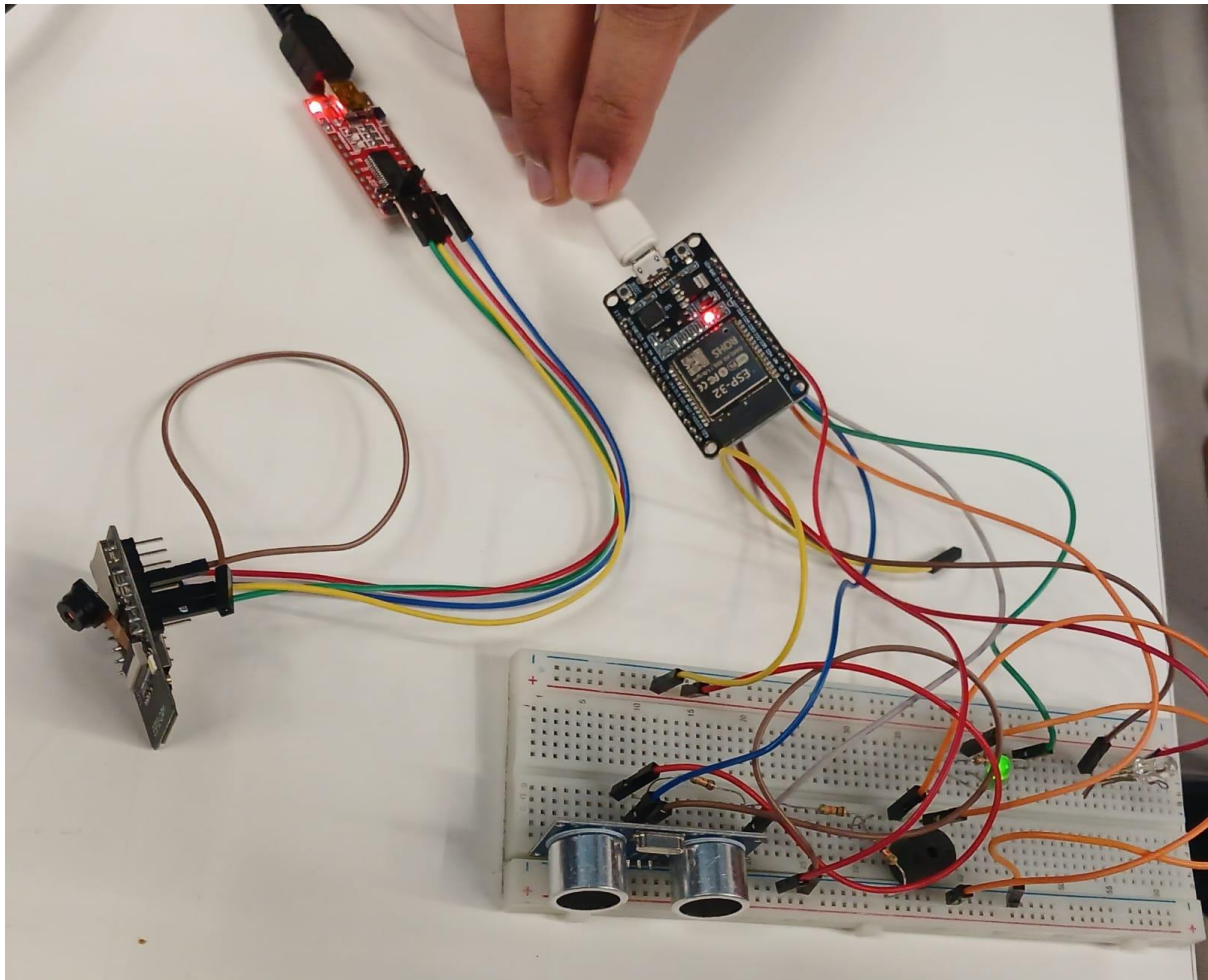
Fig. 1. Overall architecture of the IoT-based illegal parking detection system.

2. SYSTEM MODEL

2.1. Hardware Design

Every illegitimate parking spot is also watched over by a self-contained unit powered by an ESP32 microcontroller, designed for use as a smart edge node. The ESP32 microcontroller was chosen for its low cost, the presence of Wi-Fi, enough processing power, as well as its good compatibility with communication standards in IoT applications, TinyML, among others, for distributed sensing application in smart cities.

The hardware design has a modular edge computing architecture with a concentration of sensing, decision making, artificial intelligence inference, and actuation in a single location near the monitoring site.



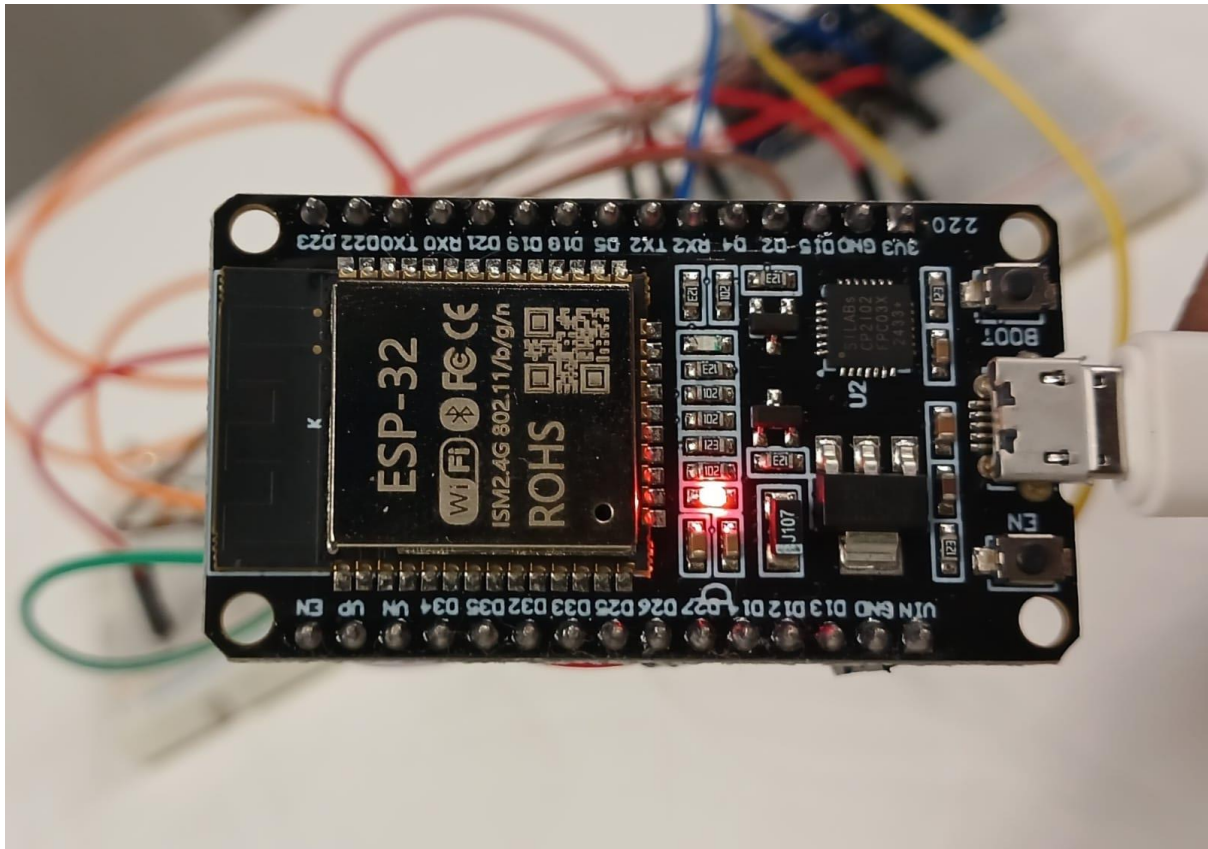
The hardware setup consists of the following main components:

ESP32 Main Module

The ESP32 acts as the central controller of the system. It is responsible for:

- Continuous acquisition of sensor data
- Executing the system decision logic
- Managing Wi-Fi and MQTT communication
- Sending commands to actuators (LEDs and buzzer)
- Exchanging verification data with the ESP32-CAM

Its dual-core processor allows reliable handling of both sensor processing and network communication.

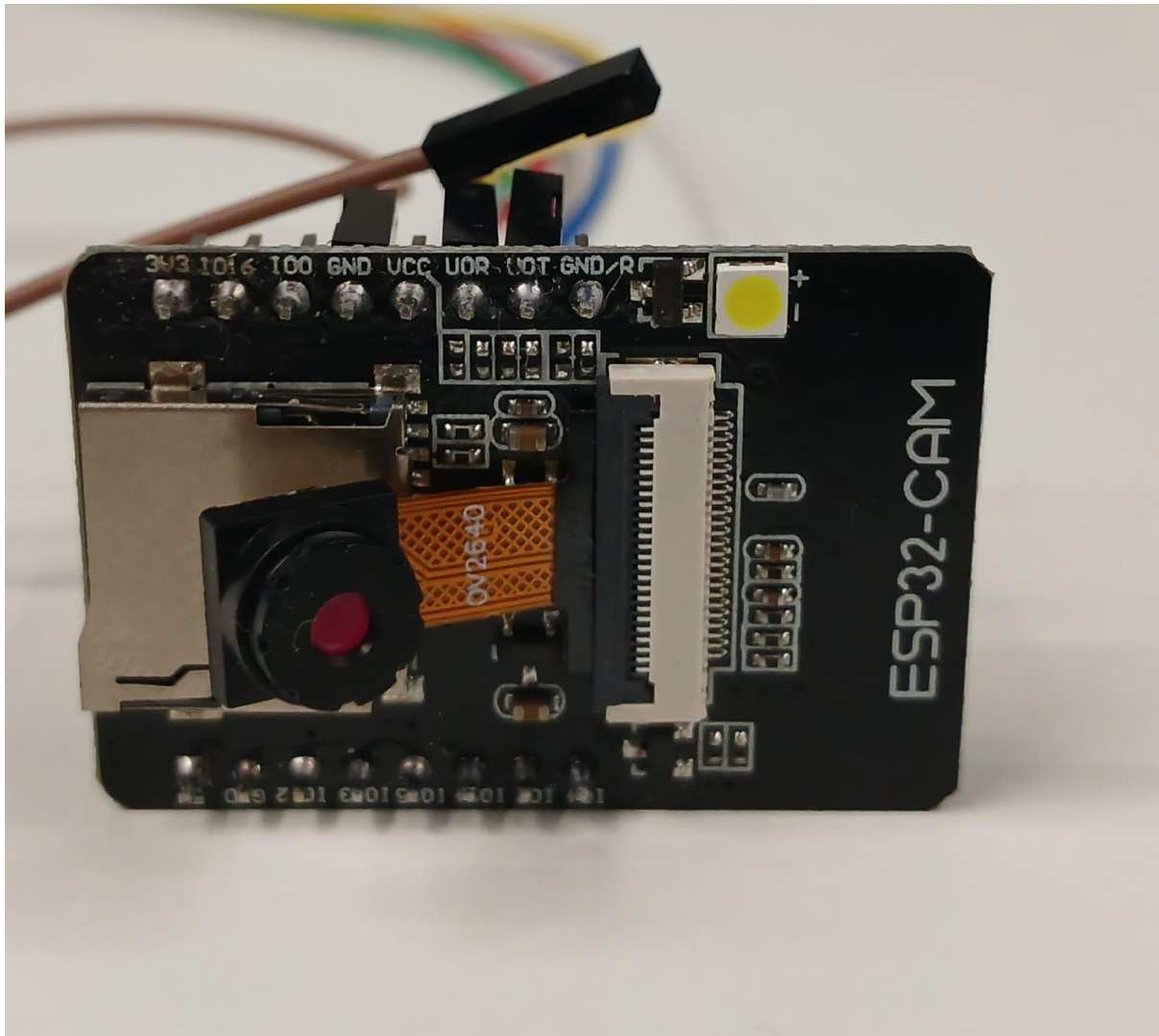


Distance / Thermodynamic Sensor

The sensor of distance continuously monitors the proximity of objects within the restricted parking zone. If a certain threshold is crossed with regards to a measured value, there exists the possibility of an obstruction. This stage serves as the first-level physical detection mechanism that allows the system to maintain energy-efficiency by only turning on advanced processes where required.

ESP32-CAM Module

The ESP32-CAM module is responsible for capturing image frames of the observed area and running an inference model using TinyML. The inference model is used to decide whether the classified object is that of a vehicle. The ESP32-CAM module sends inference results rather than transmitting unnecessary data in the form of videos since classification has been done locally. The ESP32-CAM sensor will only stream the inference results and videos in case of an observed violation.



LED Indicators

Visual indicators are used to represent system states:

- Green LED – Idle / No vehicle
- Blue LED – Object detected / Verification in progress
- Red LED – Violation confirmed

This allows for rapid recognition of status on site and facilitates debugging and maintenance.

Buzzer

The buzzer gives a sound signal when a violation has been detected. It is automatically triggered by the ESP32 and can be turned off remotely using commands on the dashboard via MQTT.

Operational Flow at Hardware Level

The distance sensor keeps on calculating the range of the monitored area. As soon as any object enters within the range, the ESP32 triggers the ESP32-CAM module to record images. As soon as the model recognizes the vehicles, if it has entered within the set time range, then it marks it as an illegal parking

case. Even then, the red LED starts lighting up, as well as the buzzer starts sounding, with the violation message sent to the server.

Each hardware component is powered and managed by the ESP32, making it a small, low power, self-sustained edge device that can run in real-time continuously.

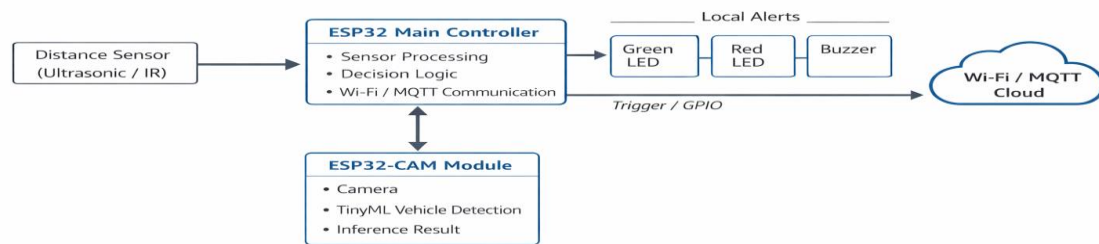


Fig. 2. Overall hardware design.

2.1.1. Hardware Control Source Code

The following code fragment demonstrates the core hardware functionality implemented on the ESP32. It handles sensor acquisition, actuator control, and serial communication with the ESP32-CAM. The program continuously monitors the distance sensor, controls LED indicators, and activates the buzzer when a potential violation condition is detected.

```
#include <WiFi.h>
#define TRIG_PIN 32
#define ECHO_PIN 33
#define RED_LED 14
#define GREEN_LED 12
#define BUZZER 25
```

```
long duration;
float distance;
```

```
void setup() {
```

```
Serial.begin(115200);
```

```
pinMode(TRIG_PIN, OUTPUT);  
pinMode(ECHO_PIN, INPUT);  
pinMode(RED_LED, OUTPUT);  
pinMode(GREEN_LED, OUTPUT);  
pinMode(BUZZER, OUTPUT);
```

```
digitalWrite(GREEN_LED, HIGH);  
digitalWrite(RED_LED, LOW);  
digitalWrite(BUZZER, LOW);  
}
```

```
float readDistance() {  
    digitalWrite(TRIG_PIN, LOW);  
    delayMicroseconds(2);  
    digitalWrite(TRIG_PIN, HIGH);  
    delayMicroseconds(10);  
    digitalWrite(TRIG_PIN, LOW);  
  
    duration = pulseIn(ECHO_PIN, HIGH);  
    return duration * 0.034 / 2;  
}
```

```
void loop() {  
    distance = readDistance();  
    Serial.print("Distance: ");  
    Serial.println(distance);  
  
    if (distance < 50) { // Object detected  
        digitalWrite(GREEN_LED, LOW);  
        digitalWrite(RED_LED, HIGH);  
        digitalWrite(BUZZER, HIGH);  
  
        // Signal ESP32-CAM for image capture / ML inference  
        Serial.println("OBJECT_DETECTED");  
    }  
}
```



```

} else {
    digitalWrite(GREEN_LED, HIGH);
    digitalWrite(RED_LED, LOW);
    digitalWrite(BUZZER, LOW);
}

delay(500);
}

```

Code Functionality Explanation

- The ultrasonic sensor continuously measures the distance to objects in the monitored area.
- If the measured distance drops below the defined threshold, the ESP32 identifies a possible parking event.
- The system activates the red LED and buzzer and sends a trigger signal to the ESP32-CAM.
- If no object is present, the system remains in idle mode with the green LED active.

2.2. Software Design

The software architecture for the illegal parking detection system involves a distributed edge-to-server model whereby functions such as time-critical detection and intelligence are performed on the ESP32 modules, and coordination and visualization are handled by the central server. This enhances scalability and real-time monitoring for several illegal parking sites.

The following five main layers can be identified in this software system:

1. TinyML inference layer (ESP32-CAM)
2. Edge control and decision layer (ESP32 main module)
3. Communication layer (MQTT, HTTP, WebSocket)
4. Backend processing and database layer
5. Frontend visualization and control layer

2.2.1. TinyML Vehicle Detection Model

In order to increase the reliability of detection and reduce the issue of false alarms, the TinyML model runs on the ESP32-CAM module. In fact, the TinyML model's main task is to identify vehicles from other irrelevant objects like pedestrians, shadows, or environmental noise. The model recognizes the captured images into two groups: when there is a vehicle and when there is no vehicle.

These frames are then processed by the TensorFlow Lite Micro machine learning engine. The model predicts a confidence value between 0 and 1. If the confidence value surpasses a threshold value, the object is then deemed to be a vehicle. This AI-enabled validation process ensures that the violation processing stage is triggered by a valid detection.

The benefits of executing the model within the device include:

- Low latency since the inference is done locally
- Lower use of bandwidth, as raw images are no longer streamed
- Increased privacy because personal data is kept at the edge
- Increased system robustness, as the scanning process is independent of continuous server connectivity

The ESP32-CAM transmits live video only when there is an actual violation and when the dashboard user wants to have visual proof.

TinyML-based Vehicle Detection Workflow on ESP32-CAM

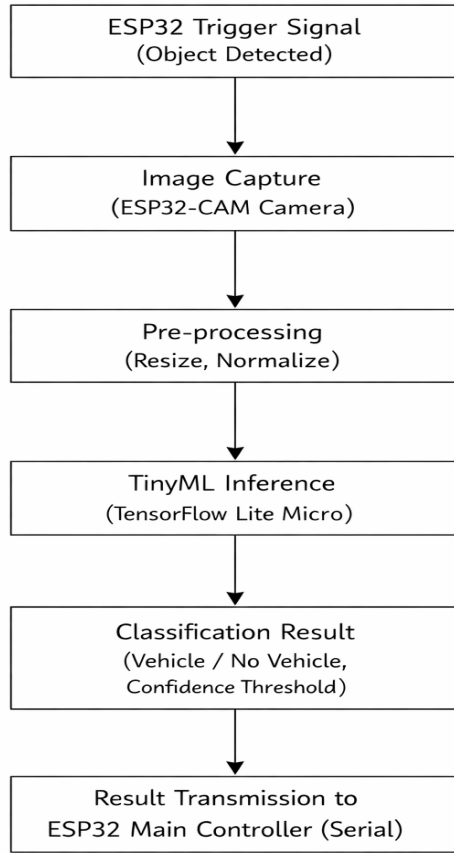


Fig. 3: TinyML-based vehicle detection workflow on ESP32-CAM.

2.2.2. Detection and Decision Logic

The primary ESP32 runs a finite-state decision engine, where it will orchestrate sensor information, TinyML feedback, and system responses. This system has four major state transitions, which are idle, object detected, vehicle confirmed, and violation.

Firstly, it continues to be in the idle state as it continuously monitors the distance and thermodynamic sensor values. But as soon as it recognizes any obstacle within the predefined threshold, it shifts to the object-detected state and sends a request to ESP32-CAM to perform image analysis.

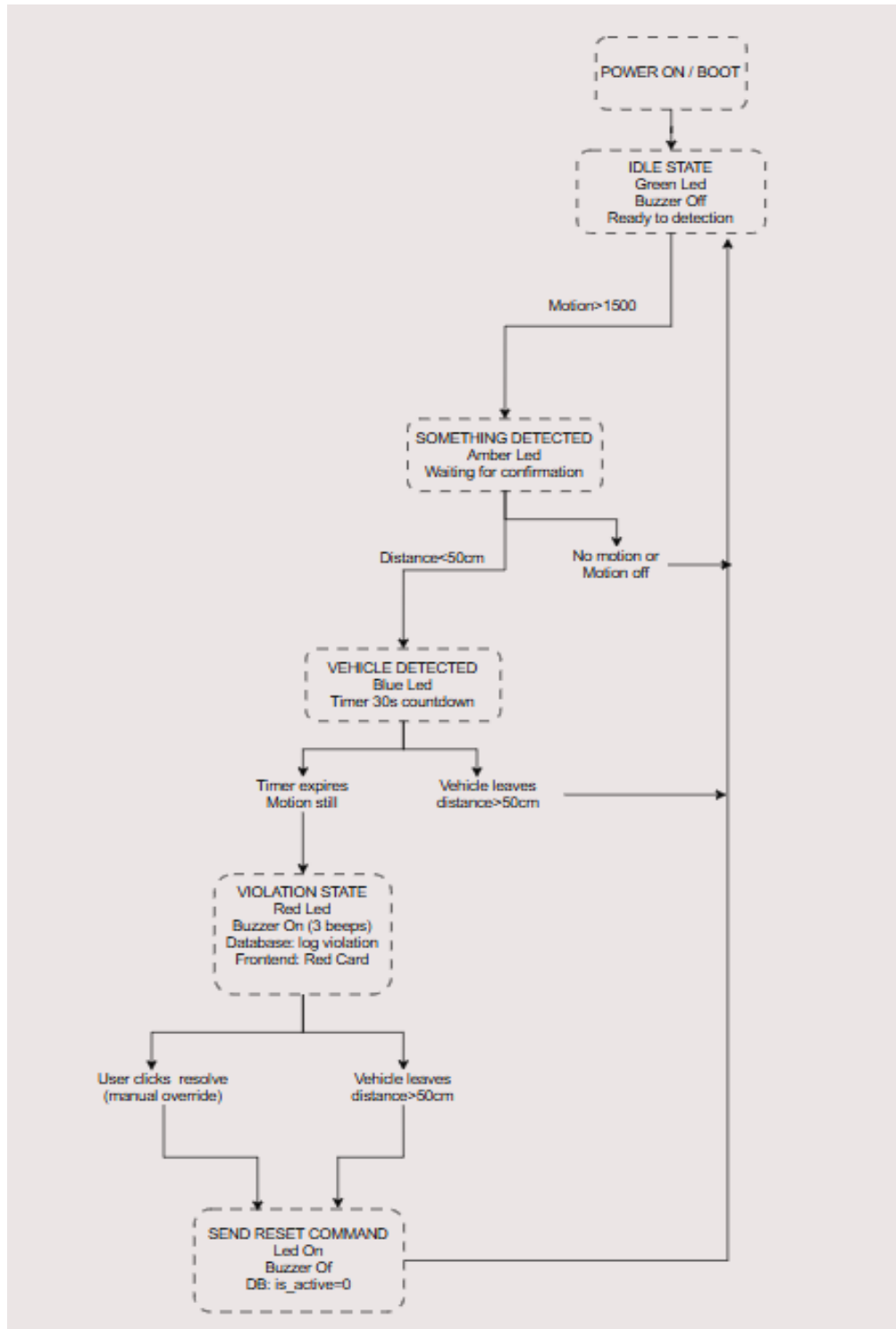
When the TinyML model verifies the existence of a vehicle, the system moves to the vehicle-confirmed state and starts a timer. The timer prevents a violation when the vehicle halts for a short period of time because, if it stays longer than allowed, it moves to the violation state.

In the violation state:

- The red LED is switched on

- The buzzer alarm is triggered
- A violation message is transmitted using MQTT
- Backend will update dashboard and log event.

This multi-layer logic ensures that a breach will only be signaled when both physically sensed and AI-validated conditions are met. Moreover, accuracy levels are substantially improved.



2.2.3. Communication Architecture

The system is based on the usage of a hybrid communication architecture, operating over a Local Wi-Fi network, which combines MQTT, HTTP, and WebSocket technologies.

- MQTT is used for lightweight real-time communication between ESP32 nodes, MQTT broker, and the backend in charge of transferring:
 - Vehicle detection results
 - Violation alerts
 - Node status updates
 - Remote control commands (e.g., buzzer silencing)
- By using the HTTP REST APIs, the following data exchange is accomplished:
 - Violation reporting
 - Session logging
 - Database updates
 - System configuration operations
- WebSocket (Socket.IO) connections are utilized to push real-time updates from the backend to the dashboard user interface.

This multi-protocol strategy enables both high responsiveness to real-time monitoring and reliable management of data during long-term system functioning.

2.2.4. Dashboard Application

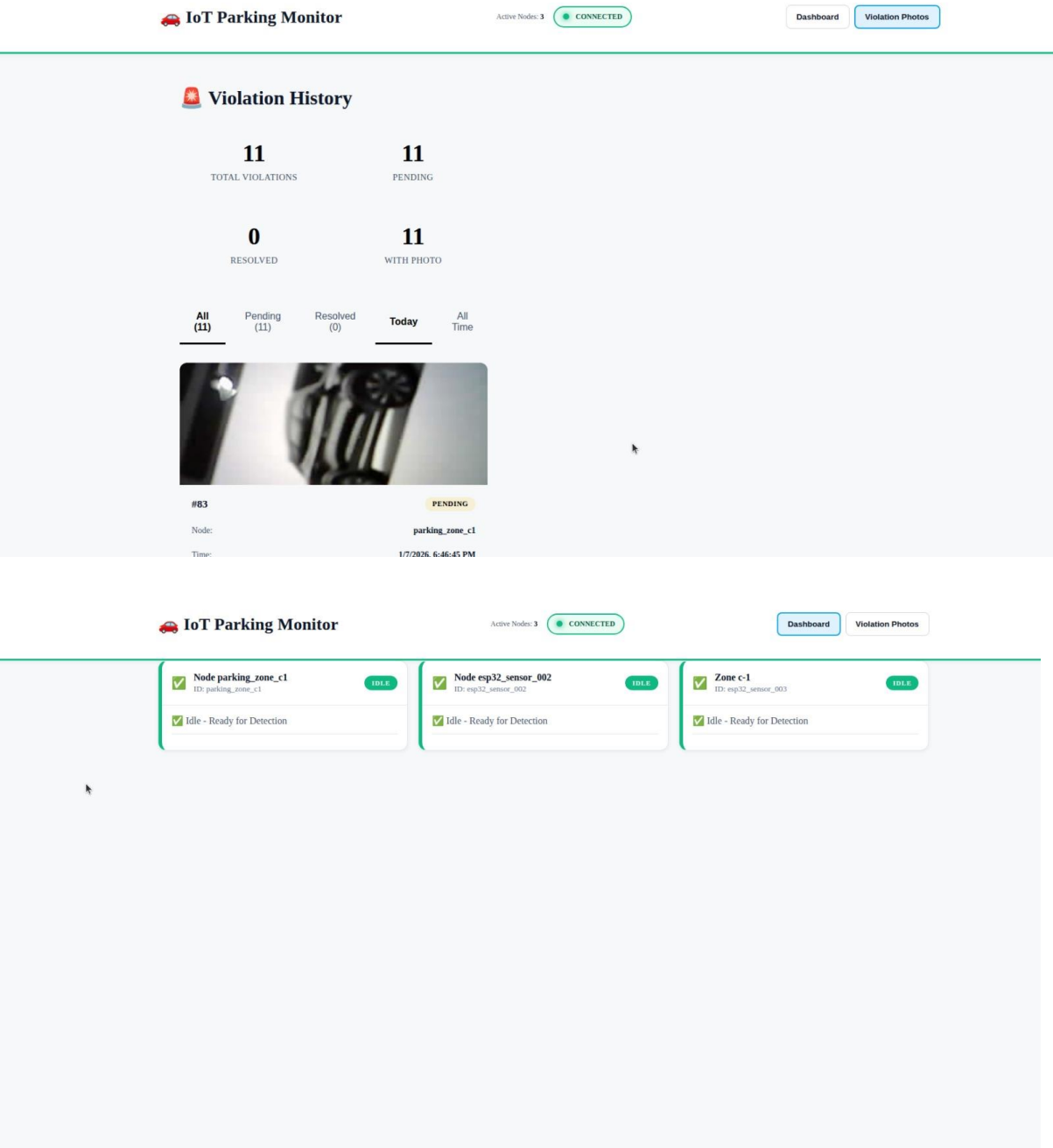
A web-operated dashboard was also developed to create an overall operation interface. The dashboard was designed to display all the deployed modules as a series of cards, with each card representing an illegal parking spot, and these cards change color dynamically to indicate the statuses.

When a violation occurs:

- The corresponding module card turns **red**
- A “**View Video**” button becomes available
- The event is logged and timestamped
- The live camera feed can be accessed for verification

The live system updates are received via WebSocket communication to the dashboard. Besides visualization, the dashboard offers remote controlling functionalities to the operators, which silence buzzers, acknowledge violations, and monitor historical activity.

It is this interface that converts the system from a sensing network into a comprehensive real-time monitoring and enforcement platform.





Violation History

1

TOTAL VIOLATIONS

1

PENDING

0

RESOLVED

1

WITH PHOTO

All
(1)

Pending
(1)

Resolved
(0)

Today

All
Time

 Violation evidence

#73

PENDING

Node:

parking_zone_c1

Time:

1/7/2026, 5:59:34 PM

Type:

parking_violation

Photo Size:

0.1 KB



2.2.5. Integration of Edge Intelligence

The integration of TinyML turns the system into an intelligent edge computing platform from a conventional sensor-based solution. Rather than sending continuous raw video streams, it sends only inference results, status updates, or violation events across the channel.

This design provides several benefits:

- Reduced network congestion
- Faster response time
- Lower server workload
- Improved scalability

Because each ESP32 node acts in an autonomous manner, the system can easily be scaled up to support dozens of illegal parking zones. In future implementation, it is easy to introduce a gateway module that aggregates the nodes, forming city-scale smart enforcement infrastructure.

3. RESULTS AND DISCUSSION

Several trials were conducted on campus to evaluate the performance characteristics, reliability, and real-time functionality of the illegal parking system.

The range sensors and thermodynamic sensors were able to detect the presence of objects in the prohibited area and trigger the first stage of the detection pipeline. The sensors were able to deliver real-time readings, and this enabled the rapid detection of potential parking events. However, the sensors faced the challenge of possible false alarms from pedestrians and noise. This problem was solved using the TinyML solution for vehicle verification.

The deployed TinyML model on ESP32-CAM worked reliably for vehicle classification. With on-device inference, the system correctly identified vehicle objects from non-vehicle objects, hence reducing false alarms by a substantial margin.

Only the detections confirmed by the model could proceed to the violation workflow. This validation stage significantly enhanced robustness and avoided unnecessary alarms in the system.

The finite-state decision logic, executed on the main ESP32 module, operated consistently throughout testing. Transitions to and from the idle, object detected, vehicle confirmed, and violation states were stable and repeatable. Temporary stops did not trigger any violations with the implemented timing mechanism, while sustained vehicle presence did correctly result in confirmed violation states. Upon the detection of a violation, the system reliably activated the red LED and buzzer alarm, providing immediate local feedback.

Communication performance was assessed with the MQTT protocol running on top of local Wi-Fi. The ESP32 nodes demonstrated the publishing of detection data, system states, and violation events to the MQTT broker. The backend system received these messages with very minor delay and relayed them forward to the dashboard. All test cases relating to dashboard updates taking place within a few seconds of any confirmed violation demonstrated near real-time responsiveness. Control commands like remote buzzer silencing also showed reliability in delivery from the dashboard down to the ESP32 modules.

The backend and database interface recorded all system events, such as sensor events, vehicle confirmations, violation dates, and resolution responses, which is useful for traceability purposes, historical records included. Data loss was not noted during the series of test cycles, which is a clear indication that the interaction between the messaging service and the database service is well integrated.

The dashboard app was able to show the actual status of all modules deployed in real time. Each parking spot was represented as a different card, and any change in status was readily observable from the change in colors. On observation of any infringement, the respective module card turned red, and an option for “View Video” appeared. The video streaming feature allowed operators to monitor infringement in videos, thereby enhancing the usability of this system.

It showed generally stable operation, low latency in responses, and consistency in the accuracy of detection. In this way, physical sensing combined with TinyML verification and distributed decision logic resulted in a truly robust solution for illegal parking detection. The obtained results confirm that the proposed architecture is suitable for real-time monitoring and can be scaled to support more than one parking location within a smart-city environment.

4. FUTURE WORK

Despite its capability to perform effectively and accurately through IoT technology for the detection of illegal parking, there are still areas for improvements that can be adopted for future development.

One of the major extensions of the work is the incorporation of automatic license plate recognition. This is because the system would be able to not only detect vehicles but also recognize their plates. This would help the system to allow the identification of offending vehicles and the subsequent issuance of fines or warning to the offenders.

Another crucial area for future improvements would be to migrate from a local deployment to a cloud-based and multi-location framework. By leveraging cloud platforms for hosting backend services, it would be possible to enable deployment at a city level or even inter-city level.

In addition, one could extend the communication infrastructure by incorporating long-range wireless communication technologies such as LoRaWAN and/or NB-IoT. This would enable deployment

within areas that do not have Wi-Fi coverage and would enable large-scale smart city deployment with low power consumption.

Finally, future research could involve the use of data analytics and predictive modeling based on the historical violation data that has been gathered. Machine learning algorithms may be employed to analyze patterns of parking behavior to pinpoint high risk areas of violations to inform urban planning. Such information will revolutionize the system from a response-driven monitoring system to a pro-active traffic control system.

In general, these future improvements would increase the system's intelligence, scalability, and real-world effectiveness, thus making the system a more integrated solution for intelligent urban transport management.

5. SOURCE CODE

Full source code and system documentation can be found at:

<https://github.com/zaidi14/comp413-g2-illegal-parking-monitor>

REFERENCE LIST

- [1] D. Shoup, *The High Cost of Free Parking*, Chicago, IL, USA: Planners Press, 2017.
- [2] A. Al-Turjman and M. Malekloo, "Smart parking in IoT-enabled cities: A survey," *Sustainable Cities and Society*, vol. 49, pp. 1–13, 2019, doi: 10.1016/j.scs.2019.101608.
- [3] S. Idris, E. Tamil, N. M. Noor, Z. Razak, and K. W. Fong, "Parking guidance system utilizing wireless sensor network and ultrasonic sensor," *Information Technology Journal*, vol. 8, no. 2, pp. 138–146, 2009.
- [4] M. Ammar, G. Russello, and B. Crispo, "Internet of Things: A survey on the security of IoT frameworks," *Journal of Information Security and Applications*, vol. 38, pp. 8–27, 2018.
- [5] P. Warden and D. Situnayake, *TinyML: Machine Learning with TensorFlow Lite on Arduino and Ultra-Low-Power Microcontrollers*, Sebastopol, CA, USA: O'Reilly Media, 2019.

